Concentric and Archimedes Chess

Version 0

Group 20

Josef Graf	grafj1@mcmaster.ca	400443948	Project Manager, Developer
Carter Naganobu	naganobc@mcmaster.ca	400471871	Developer
Roland Kim	kimr30@mcmaster.ca	400407536	Developer
Skyler Suk	suks2@mcmaster.ca	400411488	Developer
Harshmeet Singh	singh27@mcmaster.ca	400313448	Developer
Harkirat Singh	singh75@mcmaster.ca	400434064	Developer
Sparsh Poddar	poddas2@mcmaster.ca	400409187	Developer

Software and Requirement Specification..... 4

1. Project Purpose..... 4
2. Client and Other Stakeholders..... 5
3. Project Constraints..... 5
4. Functional Requirements..... 6
5. Data and Metrics..... 8
6. Non-Functional Requirements..... 9
   - 6.1 Performance Requirements:..... 9
   - 6.2 Security Requirements..... 9
   - 6.3 Reliability Requirements:..... 9
   - 6.4 Usability Requirements:..... 9

6.5 Maintainability Requirements..... 10
6.6 Compatibility Requirements..... 10
6.7 Accessibility and Scalability Requirements..... 10
7. Risks and Predicted Issues..... 10
Project Development Plan..... 12
8. Team Meeting and Communication Plan..... 12
9. Team Member Roles..... 12
10. Workflow Plan..... 13
10.1 Accomplishing Requirements..... 14
11. Proof of Concept Demonstration Plan..... 15
12. Technology..... 16
12.1 Frontend..... 16
12.2 Backend..... 16
12.3 Infrastructure..... 16
13. Project Scheduling (Gantt Chart)..... 17

Contributions

Name	**Sections**
Josef Graf	8, 9, 10, 11, 12
Carter Naganobu	2, 3, 5
Roland Kim	1, 3, 4
Skyler Suk	1, 2
Harshmeet Singh	6
Harkirat Singh	7
Sparsh Poddar	4, 10.1, 13

Definitions

Term	Definition
Crescent	The cells aligned vertically through the center of the board are crescents.
Half moon	The two crescents in the very center of the board are called half moons.
Twin cell	The two horizontal cells an edge away from a crescent or half moon are twin cells.
King's initial leap	On its first move the king may leap by 2 cells, even if in check, including a knight's move. An intermediary cell may be occupied but not threatened.
Prince	A chess piece identical to a king, but not subject to check.
Berolina pawn	A chess piece similar to a normal pawn, but can move on a diagonal and capture going straight.
Elo	https://en.wikipedia.org/wiki/Elo_rating_system
Glicko	https://en.wikipedia.org/wiki/Glicko_rating_system
CSRF	Cross site request forgery, https://owasp.org/www-community/attacks/csrf
CI/CD	Continuous integration and continuous deployment, https://en.wikipedia.org/wiki/CI/CD
WCAG 2.1	https://www.w3.org/TR/WCAG21

Software and Requirement Specification

1. Project Purpose

Our project purpose is to design and complete a fully functional web-based, multiplayer implementation of two chess variants: Concentric Chess and Archimedean Chess.

Both games are two of many historical and modern variations on the traditional game of chess, featuring each their individual differences on rules, movement, pieces, and the board itself. In particular, the boards take the shape of concentric circles and an Archimedean spiral respectively, which poses an interesting challenge in the implementation and structuring of the background logic and design process.

Players of either variant may find interesting strategies and interactions that are not present in the regular game of chess, which can result in new shifts in widely-accepted moves and processes that can be overturned in a change of rules. In particular, existing players of any skill level in chess may find functional variants of a historical game both challenging and intriguing, which forms the overall playerbase and demand for easily accessible and playable chess variations.

Careful consideration must be upheld to make sure the end product is both comprehensible and functional, allowing those either familiar or unfamiliar with regular chess to be able to largely understand the rules at play and are able to play the game properly in a two-person setting. Group members will work together to communicate and divide tasks equivalently such that the overall management of the project leads it to be completed within the deadlines allotted.

2. Client and Other Stakeholders

Primary:

The primary stakeholders of the project are chess and chess variant enthusiasts, our supervisor, Professor Rapoport (rapoport@mcmaster.ca) and us. Course stakeholders, Professor Moradi, Amirhossein Sabour, and Sahib Khokar, are also primary stakeholders.

Secondary:

The secondary stakeholders of the project include more casual chess players, as well as future developers who might repurpose the ruleset, assets, or code to create new variants.

Tertiary:

The tertiary stakeholders of the project include other chess game services.Other chess game services may wish to clone the variants or work with us to implement them into their own product.

3. Project Constraints

Both variants of chess within the scope of the project will be implemented as a website and are selectable depending on which variant is desired to play. The website should be accessible to anyone with a stable internet connection, and the domain must remain playable up to and after the year the capstone project is completed.

The end product of the project must include online multiplayer support, defined as being able to connect with a separate player within the same game and are able to play each other to determine a winner. Moves and move history should be visible regardless of each player's position on the board, such that the play is synchronized between player sessions, and that the match is trackable for either player.

The proper rules respective to each variant of chess are clearly defined as provided by the professor, and are fully implemented; there should be no illegal moves possible within the application, and the rules currently functional are able to be extended or adjusted logically for alternate board states and move options, depending on the pieces in question.

As the rules can be quite unintuitive, there will be different settings for players of differing levels of familiarity with the variants. Players should have settings to enable/disable move confirmation, enable/disable the chess clock. Additionally, players will have the legal moves shown and have a rulebook available during a match.

4. Functional Requirements

P0:

1. Server-authoritative validation: All move validation (including check, checkmate, stalemate, repetition rules, and special piece powers such as king's initial leap, prince movement, crescent/half-moon behavior, twin-cell interactions) is computed server-side; the client is a visualizer only. A move is applied only if the server accepts it.
2. Deterministic rules engine: The rules for both variants must be encoded as pure, deterministic functions accepting current state → (next state | illegal) to ensure reproducible results and unit testing.
3. Desync handling: If a client state diverges from the server, the UI must auto-refresh to the last server-committed state and display a non-blocking banner indicating resynchronization.
4. Draw & termination conditions: Server detects 3-fold repetition, 50-move (or variant-appropriate) rule, impossible checkmates, and inactivity timeouts; game is closed with final result stored.

5. Legal-moves overlay: On select, the client must display all legal destinations for the piece (including variant-specific “swing”/twin-cell moves and Berolina-style pawn behavior when on crescents/half-moons).
6. Single-source clocks (if enabled): Turn timers are tracked on the server; client clocks are cosmetic mirrors and must tolerate drift correction.

P1:

1. Invite flow w/ lobby: Invitation links open a lobby that shows variant, time controls, and readiness state. The game starts only after both players confirm.
2. Surrender/Draw workflow: Requests are server-mediated and rate-limited; both are logged with timestamps and result updates.
3. Game archival format: Store games as {variant, initial_setup_id, move_list, timestamps, result, metadata} such that a replay can be reconstructed deterministically even if the ruleset evolves (versioned rules).
4. Tutorial & inline hints: Contextual tooltips (e.g., on crescents, twin cells, half-moons) can be toggled; a rules page is available in-game.

P2:

1. Matchmaking queue: Anonymous or rated queue (if rating exists) that pairs by variant and optional region.
2. Move confirmation mode: Optional “confirm to move” step with pre-move highlight; disabled/enabled per user preference and persisted to the account.
3. Replay viewer: Step-through playback, jump-to-move, and shareable replay links; supports both variants’ board renderings.

P3:

1. Rating model: Simple Elo-style or Glicko-lite per variant; unrated games do not affect rating.

2. Spectator mode: Read-only, delayed by N seconds (configurable) to reduce ghosting; spectators see clocks and moves but cannot chat with players (optional).
3. Power users: Keyboard shortcuts for navigation (confirm move, cancel, next/prev move in replay) and accessibility actions (focus cycling).

5. Data and Metrics

We will store user data relating to their accounts. This will include information they provide such as their email, password, and username in addition to any information we may gather about them as they use the product. Information we may gather will include a skill rating, and a history of all the games they play.

Likewise, game states and history will need to be stored. We will store the game information as a sequence of moves. Which can be simulated starting from a known initial set up to achieve a particular board state of the match. As well as storing related metadata and timestamps of the game.

Important to the project is that only legal moves can be made and that all legal moves can be made. To quantify this, we will create test cases for all possible situations and report the percentage of passing tests. We expect a passing rate of 100%.

For the website itself, we will follow Chrome's Lighthouse scoring. Lighthouse includes metrics such as First Contentful Paint, Speed Index, Largest Contentful Paint, Total Blocking Time, and Cumulative Layout Shift. The Lighthouse score is a weighted average of these metrics. We expect to reach a Lighthouse score of at least 75.

Currently there is no immediate plan to develop an AI to play the variants.

6. Non-Functional Requirements

6.1 Performance Requirements:

1. Move latency must be below 150ms for users in Ontario under normal network conditions
2. Board state should propagate in real time for both players.
3. Time to Interactive (TTI) must be under 3 seconds on standard broadband connection.
4. Legal moves calculation should be fast and complete within 100ms for a given board state.
5. The application must support at least 100 concurrent games without any performance drop off.

6.2 Security Requirements

1. All user passwords must be hashed using industry standard algorithm
2. Authentication tokens must be securely managed and expire appropriately.
3. Game state validation must occur server side to prevent cheating or manipulation.
4. Database access controls must prevent unauthorized access to the data.

6.3 Reliability Requirements:

1. Application must maintain uptime during expected usage periods.
2. Game state must be persistently stored to prevent data loss from server crashes or disconnections.
3. In case of client and server desynchronization, the user must be immediately notified.

6.4 Usability Requirements:

1. The application must be user-friendly, intuitive to navigate for anyone familiar with standard chess without need of any external tutorials.

2. Visual feedback for user actions including piece selection, illegal moves and such must be provided.
3. Error messages must clearly indicate the problem and suggest corrective action if applicable.
4. Include a tutorial and rules section that can be accessed from any page.

6.5 Maintainability Requirements

1. Code must follow consistent TypeScript and React best practices enforced by ESLint.
2. The critical game logic functions must have unit test coverage of at least 80%.
3. API endpoints and other major components must include inline documentation.

6.6 Compatibility Requirements

1. The application must be compatible with the current browsers including Chrome, Firefox, Safari, Edge, etc.
2. Must support various screen sizes ranging from the 375 px of a phone's screen to a 2560px desktop.
3. It must adjust to various screen sizes with suitable layouts.

6.7 Accessibility and Scalability Requirements

1. Must support horizontal scaling as user base grows
2. Must handle increased traffic without manual intervention
3. Should be accessible to everyone and must include colour contrast ration meeting WCAG 2.1 LEVEL AA standards (minimum 4.5:1 for the text)
4. Game state indicators must not rely solely on colour differentiation
5. Keyboard navigation including shortcuts and basic controls must be supported for the core gameplay actions.

7. Risks and Predicted Issues

1. The ruleset for both variants is subject to change and refinement through

playtesting. The circular board layout introduces unique neighbor relationships, including crescents, twin cells, and diagonal paths, which make validating moves across all board states prone to numerous edge cases. Edge cases and special move interactions may require clarification after implementation.

2. Different network conditions, including latency or disconnections, could cause desynchronization between clients and the server. This may lead to players seeing different board states due to race conditions. It will be a challenge to maintain synchronization, and one possible solution could involve using server-authoritative validation, Supabase real-time features, and automatic reconnection handling logic.

3. There is a possibility of client-side manipulation that could allow illegal moves. This presents a risk to server integrity and game fairness. To mitigate this, all game logic should be validated server-side, and Supabase's security features may be used if necessary.

4. User experience with the circular board may initially be confusing, making it a challenge to implement complex game logic while keeping the interface simple and intuitive. This can be addressed through clear visual indicators and, if possible, a short tutorial section to guide new users.

5. After the capstone project concludes, the application must remain functional and accessible. However, dependencies such as Cloudflare, Supabase, and third-party libraries may introduce breaking changes over time, and the team may not be available for ongoing maintenance.

Project Development Plan

8. Team Meeting and Communication Plan

Our team primarily communicates through a dedicated Discord server with separate channels for various topics including scheduling meetings, deadlines, announcements, and general discussion. Communication with our supervisor currently takes place over email, although we may transition to Discord for that as well.

Meetings will typically take place over Teams or Discord, with a preference for Wednesday evenings as availability dictates. Meetings with our supervisor will be in person on Wednesday afternoons if possible, otherwise over Teams.

All documents are drafted on Google Docs, allowing team members to seamlessly collaborate during meetings or when working.

For task and project management, we utilize GitHub Issues and Projects. Each issue corresponds to a specific task or feature, allowing direct links to associated pull requests for easy tracking. In addition, GitHub Projects allow us to coordinate milestones, distribute work across team members, and track task statuses to ensure accountability and transparency throughout the project.

9. Team Member Roles

Team Member	**Role**	**Responsibilities**
Josef Graf	Project manager, authentication and infrastructure lead	Authentication, user management, infrastructure (database, realtime, deployment), backend
Carter Naganobu	Board interface lead	Board, piece and move display, back end integration

Roland Kim	Backend and frontend developer	Invitations, game history
Harshmeet Singh	UI design and component lead	UI
Harkirat Singh	Testing and QA engineer	Testing & QA
Sparsh Poddar	Backend developer	Backend
Skyler Suk	Backend and frontend developer	Invitations, game history, frontend

10. Workflow Plan

All work will be done through GitHub following a branch-based workflow. Each feature will be developed on its own branch with a corresponding draft Pull Request (PR). Once work is completed, the PR will have its status updated and will be reviewed by 1-2 teammates before being merged into the master branch. GitHub Issues and Projects will be used to manage task and feature assignments, as well as track the state of the feature and its dependencies through their lifecycle.

We aim to follow agile principles through short sprints, holding planning meetings as needed and utilizing GitHub Projects and its kanban board to track tasks through sprints and epics.

All data will be stored in Supabase, which also handles the core infrastructure for authentication and realtime functionality. All compute tasks and application hosting will be handled by the Cloudflare Developer Platform.

Performance goals, including latency and responsiveness, will be met through use of Cloudflare's edge network, which uses compute resources in Ontario to minimize load times for core users, and through our Supabase project's placement in Ontario.

10.1 Accomplishing Requirements

- P0.1 Legal move generation & display → Next.js (TS) UI, Rules Engine in server routes (Next.js Route Handlers), unit tests for rules (Vitest or Node test), Cypress E2E to click-select and assert overlays.
- P0.2 Illegal-move prevention → Server-authoritative apply; client dispatches an intent, server replies with accept/reject; UI blocks on "in-flight" and rolls back on reject.
- P0.3 Live board state → Supabase Realtime channels broadcast committed moves; UI subscribes to room topic; optimistic UI disabled for correctness.
- P0.4 Desync detection → Each state carries a monotonic version (or hash). On mismatch, client triggers full state fetch; toast/banner shows "synced to server."
- P0.5 Game termination → Server computes checkmate/stalemate/repetition/timeouts; writes {result, reason}; triggers final broadcast and persistence.
- P1.1 Accounts → Supabase Auth (email/password + OAuth), password hashing managed by Supabase; session on edge (Cloudflare).
- P1.2 Invites → Server creates game_invite {variant, options, expires_at, token}; client shares deep link /invite/:token; lobby uses Supabase Realtime to show readiness.
- P1.3 Surrender → POST /games/{id}/surrender (server verifies turn or allow-anytime policy), persists result; UI shows modal confirmation.
- P1.4 Draw offer → POST /games/{id}/draw-offer, server stores pending; opponent POST accept/decline; rate-limited by server.
- P1.5 Game storage → Postgres schema: games, game_moves, game_meta. Store rules_version and initial_setup_id to ensure future-proof replays.
- P2.1 Matchmaking → Simple queue table; background worker (Edge function / Cron) matches by variant & matchmaking ranking (if available).

- P2.2 Move confirmation → Client state machine (idle → selecting → confirm), cancellable; persisted user preference; E2E tests with Cypress.
- P2.3 Replays → Read model from game_moves; React viewer with scrubber; shareable URL; no auth required for public games (configurable).
- P3.1 Ratings → player_ratings (player_id, variant, rating, rd, last_updated); update on results; display on profile and matchmaking.
- P3.2 Spectators → Read-only subscription channel /games/{id}:spectate; server can buffer N-second delay; limit spectators by policy.
- P3.3 Shortcuts → Client keymap; ARIA-compliant focus management; unit tests for handlers.

Cross-cutting:

- Testing: Unit (rules), integration (move apply), end to end (Cypress flows).
- Observability: Sentry for client+server errors; logs on move rejects and desync events.
- Performance: Bench legal-move calc; target <100ms per state; Lighthouse ≥75.
- Security: Row-level security in Supabase; server only trusts DB-verified user/session; all state-changing ops require CSRF-safe POSTs.

11. Proof of Concept Demonstration Plan

The primary risk in our project lies in handling the wide variety of edge cases introduced by the circular board layout and the additional complexities of it compared to the classic chess game. To address this, our proof of concept will demonstrate the technical feasibility of representing and processing the game logic. We will showcase a simple website that showcases a basic version of an interactive board capable of displaying and validating legal moves, as well as a minimal backend to manage some game state. In addition, we will include a rules page and placeholders for future features such as matchmaking, game history, and profile management.

12. Technology

12.1 Frontend

We will use TypeScript for our frontend language, utilizing React and the Next.js App Router as our frontend framework. We also plan to use TailwindCSS and shadcn/ui as our UI libraries and ESLint for linting. Given the complexity of the application and the number of edge cases in the ruleset, we also plan to implement end to end and component testing using the Cypress testing framework, unit testing using Vitest, and error monitoring using Sentry.

12.2 Backend

We will again use TypeScript as our backend language, making use of Next.js Server Actions and Route Handlers to handle all requests. We also plan to use Supabase (built on Postgres) for our database and to handle the core infrastructure for authentication and realtime, as well as Cypress, Vitest, and Sentry for testing and error tracking.

12.3 Infrastructure

We will be hosting our application on the Cloudflare Developer Platform to make use of their edge network and all data, authentication, and realtime processing will be hosted using Supabase. We will use GitHub Actions as our CI/CD runner, with preview and production pipelines. The preview pipeline will run on each PR and commit, running the linter, all tests, and deploying a preview site to Cloudflare using a preview branch of our Supabase instance. The production pipeline will run on every commit to the master branch, running the linter, all tests, and deploying the production version of the site to Cloudflare.

13. Project Scheduling (Gantt Chart)

